Fichas bibliográficas sobre sistemas distribuidos

FORM-FB-01

García Bazurto Kevin

Ficha 1 Consistencia en Kubernetes

[1] J. Redžepagić et al., «Empirical performance and operational analysis of monolithic and distributed database architectures in kubernetes environments,» *Computers*, vol. 15, n.º 5, 29 de abr. de 2026, art. 282. DOI: 10.3390/computers15050282
DOI 10.3390/computers15050282

Resumen

En Kubernetes, el rendimiento de una base de datos depende tanto del motor como de su despliegue. El estudio compara PostgreSQL, MySQL, MongoDB, Redis y Cassandra con herramientas de carga específicas, variando la concurrencia y las operaciones. Para Cassandra, contrasta escrituras con ONE y QUORUM y registra rendimiento, latencias y dispersión entre ejecuciones. En la configuración probada, QUORUM reduce la tasa media de escritura y aumenta su variabilidad. La comparación permite distinguir la rapidez habitual de los retrasos que aparecen en una fracción de las peticiones. Los resultados corresponden a ese clúster y requieren controlar las condiciones al intentar reproducirlos.

Dato verificable

Tabla 20, filas de escritura. La media fue de 157.477 operaciones por segundo con ONE y de 86.340 con QUORUM, los grupos tuvieron cuatro y cinco ejecuciones.

Nota personal

Este estudio ayuda a comprender por qué una escritura puede tardar más cuando Cassandra necesita confirmar una operación con más réplicas. Si eligiera Cassandra, lo usaría en la sección de “Experimentación propia” para justificar el rendimiento como se indica en el percentil 99 realizado en el estudio al comparar ONE y QUORUM.

Ficha 2 Escalado y coordinación entre réplicas

[2] H. Nejati Sharif Aldin y F. Nayebi Pour, «From watts to writes: The role of frequency, replication strategies and consistency levels in performance and energy-aware cassandra cluster operations,» *Future Generation Computer Systems*, vol. 176, mar. de 2026, art. 108210. DOI: 10.1016/j.future.2025.108210
DOI 10.1016/j.future.2025.108210

Resumen

Ahorrar energía en Cassandra requiere entender cómo interactúan la configuración del clúster y la del hardware. Los autores varían la frecuencia del procesador, la replicación, el nivel de consistencia y las cargas de YCSB (Yahoo! Cloud Serving Benchmark). Con un modelo de potencia, estiman el consumo dinámico y seleccionan configuraciones que equilibran energía y tiempo de ejecución dentro de un límite de latencia. Al añadir nodos, el rendimiento también mejora con ONE y replicación completa, mientras que con ALL la coordinación puede aumentar la latencia. Así, el efecto de ampliar el clúster depende de cuántas respuestas debe esperar cada operación.

Dato verificable

Sección Abstract. Al pasar de 10 a 20 nodos, la latencia de red aumentó hasta un 90 % bajo ALL en las pruebas que se reportaron.

Nota personal

Este artículo ayuda a entender que añadir nodos puede aumentar el rendimiento y también empeorar en el tiempo de respuesta al usar ALL porque tarda en esperar respuesta de todas las réplicas. Este estudio lo usaría en el análisis de PACELC para discutir el costo de esperar todas las réplicas incluso cuando la red funciona.

Ficha 3 Consistencia en varias regiones

[3] S. Ferreira et al., «Benchmarking Consistency Levels of Cloud-Distributed NoSQL Databases Using YCSB,» *IEEE Access*, vol. 13, págs. 63 428-63 438, 2025. DOI: 10.1109/ACCESS.2025.3558923
DOI 10.1109/ACCESS.2025.3558923

Resumen

La replicación entre regiones protege el acceso a los datos, pero esperar confirmaciones remotas puede reducir el rendimiento. Los autores comparan Cassandra, MongoDB y Redis en un entorno multirregional de nube. Aplican YCSB para ejecutar cargas con distintas proporciones de operaciones y contrastan configuraciones de consistencia menos exigentes con otras más exigentes. En Cassandra encuentran que algunas cargas procesan muchas menos operaciones por segundo cuando se eleva la exigencia de confirmación. También reportan aumentos del tiempo de ejecución en Redis. El valor del estudio está en mostrar cuánto puede variar el costo según el motor, la carga y la configuración.

Dato verificable

En la carga A de YCSB, Cassandra pasó de procesar 868,20 a 37,84 operaciones por segundo al cambiar de la configuración de consistencia débil a la fuerte lo cual representa una reducción del 95,64 %.

Section V, Experimental Results, A. Impact of Different Workloads.

Nota personal

Este estudio me ayuda a entender por qué esperar la confirmación de réplicas ubicadas en distintas regiones puede reducir la cantidad de operaciones que se procesan por segundo. Para el TA-IND-01, usaría los resultados de la carga A de YCSB en el análisis comparativo para explicar y sustentar cómo cambió el rendimiento de Cassandra y MongoDB al pasar de la configuración de consistencia débil a la fuerte.

Ficha 4 Rendimiento según el despliegue

[4] S. Ferreira et al., «Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications,» *Journal of Cloud Computing*, vol. 13, n.º 1, 27 de nov. de 2024, art. 158. DOI: 10.1186/s13677-024-00716-7
DOI 10.1186/s13677-024-00716-7

Resumen

El estudio examina cómo influye la consistencia en el rendimiento de bases NoSQL desplegadas en una región o en varias. Compara Cassandra, MongoDB y Redis bajo diferentes niveles de concurrencia. Los experimentos con Locust registran solicitudes por segundo, tiempos de respuesta y errores, mientras la monitorización de nube aporta datos de recursos. En Cassandra, las pruebas comparan ONE y ALL con tres réplicas. Dentro de una región, algunos escenarios de lectura y escritura pierden rendimiento con la configuración más exigente. La separación por operación y topología permite formular comparaciones entre motores y entender por qué cambia el costo de coordinación.

Dato verificable

En el despliegue de una sola región, con dos usuarios concurrentes, Cassandra pasó de procesar unas 1500 a unas 970 solicitudes de escritura por segundo al pasar de la configuración de consistencia débil a la fuerte. Esto representa una reducción cercana al 36 %. El resultado aparece en la figura 2 y en el párrafo sobre el rendimiento de escritura de la subsección Single-region deployment.

Nota personal

Esta comparación ayuda a entender por qué conviene analizar las lecturas y las escrituras por separado ya que cada una afecta en el rendimiento de respuesta de estas bases de datos, para el TA-IND-01, usaría la figura 2 en el análisis de PACELC para mostrar cómo cambió el rendimiento de escritura al aumentar la exigencia de consistencia en una sola región con dos usuarios concurrentes.

Ficha 5 Replicación en equipos de bajo consumo

[5] L. F. da Silva y J. V. F. Lima, «An evaluation of relational and NoSQL distributed databases on a low-power cluster,» *The Journal of Supercomputing*, vol. 79, n.º 12, págs. 13 402-13 420, ago. de 2023. DOI: 10.1007/s11227-023-05166-7
DOI 10.1007/s11227-023-05166-7

Resumen

Los autores estudian si un clúster económico de placas de bajo consumo puede sostener bases de datos distribuidas. Despliegan Cassandra, HBase y Citus sobre PostgreSQL mediante Docker Swarm en quince Raspberry Pi. Con cargas YCSB comparan el rendimiento al variar el factor de replicación. En su entorno, Cassandra obtiene ventajas cuando el cliente puede especificar el nivel de consistencia. En Citus y HBase, aumentar las réplicas penaliza el rendimiento observado. La prueba ayuda a distinguir la cantidad de copias de la política que decide cuándo responde una operación y muestra por qué ambas variables deben declararse en una comparación experimental.

Dato verificable

Los autores realizaron las pruebas en un clúster de 15 nodos Raspberry Pi 3 y usaron Docker Swarm para desplegar los servicios. Este dato aparece en la sección 1, “Introduction”.

Nota personal

Este estudio ayuda a distinguir el número de copias que guarda el sistema de las respuestas que exige para completar una operación. Lo usaría en Tolerancia a fallos para explicar esa distinción y en Experimentación propia para justificar que mantenga fijo el factor de replicación al cambiar el nivel de consistencia.

Declaración de uso de inteligencia artificial

Utilicé Codex (OpenAI, versión 6.1 Sol) como apoyo para buscar fuentes académicas confiables y mejorar la redacción del trabajo.

Referencias

[1] J. Redžepagić, A. Kapulica, N. Malešević y V. Dakić, «Empirical performance and operational analysis of monolithic and distributed database architectures in kubernetes environments,» *Computers*, vol. 15, n.º 5, 29 de abr. de 2026, art. 282. DOI: 10.3390/computers15050282

[2] H. Nejati Sharif Aldin y F. Nayebi Pour, «From watts to writes: The role of frequency, replication strategies and consistency levels in performance and energy-aware cassandra cluster operations,» *Future Generation Computer Systems*, vol. 176, mar. de 2026, art. 108210. DOI: 10.1016/j.future.2025.108210

[3] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo y E. Andrade, «Benchmarking Consistency Levels of Cloud-Distributed NoSQL Databases Using YCSB,» *IEEE Access*, vol. 13, págs. 63 428-63 438, 2025. DOI: 10.1109/ACCESS.2025.3558923

[4] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo y E. Andrade, «Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications,» *Journal of Cloud Computing*, vol. 13, n.º 1, 27 de nov. de 2024, art. 158. DOI: 10.1186/s13677-024-00716-7

[5] L. F. da Silva y J. V. F. Lima, «An evaluation of relational and NoSQL distributed databases on a low-power cluster,» *The Journal of Supercomputing*, vol. 79, n.º 12, págs. 13 402-13 420, ago. de 2023. DOI: 10.1007/s11227-023-05166-7